CSA4704-DEEP LEARNING

NAME:HARITHI S **REG NO:192421151**

Experiment 20: Probabilistic PCA and Reconstruction Analysis

Problem Statement: A sensor network in a smart factory produces readings with random noise and occasional missing values. Engineers want a model that accounts for noise explicitly. Multivariate sensor readings (temperature, vibration, pressure, current and so on) with added Gaussian noise. The objective is to apply probabilistic PCA and compare its reconstruction with standard PCA. Use NumPy, Scikit-learn,Matplotlib.

Tasks:

1. Explain the PPCA generative model $x = Wz + \mu + \varepsilon$.

2. Fit PPCA (Scikit-learn PCA with its noise-variance estimate, or an EM implementation in NumPy).

3. Estimate the noise variance $\sigma 2$.

4. Reconstruct the data and compute reconstruction error against component count.

5. Generate new synthetic sensor samples from the fitted model.

FORMULA:

1. PPCA Generative Model

$X = WZ + \mu + \varepsilon$

2. Gaussian Noise

$\varepsilon \sim N(0, \sigma^2 I)$

3. Latent Variable Distribution

$Z \sim N(0, I)$

4. Noise Variance Estimation

σ^2 = (Sum of Remaining Eigenvalues) / (D - K)

5. Reconstruction Formula

$X_{reconstructed} = Z \times W.T + \mu$

6. Reconstruction Error

MSE = Sum((X - $X_{reconstructed}$)²) / Number of Elements

7. Mean Calculation

μ = (Sum of all data values) / Number of samples

8. PPCA Covariance Matrix

$C = W \times W.T + \sigma^2 I$

9. Generate New Samples

$X_{new} = W \times Z_{new} + \mu + \varepsilon_{new}$

10. Standard PCA Reconstruction

$X_{reconstructed} = X_{reduced} \times Eigenvectors.T + Mean(X)$

CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.decomposition import PCA

from sklearn.preprocessing import StandardScaler

np.random.seed(42)

# 1. Generate synthetic smart-factory sensor data

n_samples = 1000

# Hidden factors affecting the sensors

Z_true = np.random.randn(n_samples, 2)

W_true = np.array([

    [2.0, 0.5],

    [1.5, 0.3],

    [0.2, 2.0],

    [0.5, 1.5],

    [1.0, 0.8]

])

sensor_mean = np.array([25, 5, 100, 10, 60])

# Generate sensor readings and add Gaussian noise

X_clean = Z_true @ W_true.T + sensor_mean
```

```
noise_std = 0.5
X = X_clean + np.random.normal(0, noise_std, X_clean.shape)
sensor_names = [
    "Temperature", "Vibration", "Pressure",
    "Current", "Humidity"
]
print("Sensor data shape:", X.shape)
# Standardize sensor features
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
# 2. Fit standard PCA and estimate noise variance
max_components = X.shape[1] - 1
print("\nNOISE VARIANCE ESTIMATION")
print("-" * 40)
for k in range(1, max_components + 1):
    pca = PCA(n_components=k)
    pca.fit(X_scaled)
    print(f"Components: {k}, Estimated noise variance: "
          f"{pca.noise_variance_:.4f}")
# Fit a two-component model for detailed analysis
```

```
k = 2

pca = PCA(n_components=k)

pca.fit(X_scaled)



sigma2 = pca.noise_variance_

mu = pca.mean_

print("\nSelected components:", k)

print("Estimated noise variance (sigma^2):",

    round(sigma2, 4))

print("Estimated noise standard deviation:",

    round(np.sqrt(sigma2), 4))

# 3. Build the PPCA loading matrix W

# PPCA: x = Wz + mu + epsilon

# W is constructed from PCA eigenvectors and eigenvalues.

eigenvalues = pca.explained_variance_

eigenvectors = pca.components_.T

adjusted_values = np.maximum(eigenvalues - sigma2, 0)

W = eigenvectors * np.sqrt(adjusted_values)

print("\nPPCA loading matrix shape:", W.shape)

# 4. Reconstruct data using standard PCA and PPCA
```

```
# Standard PCA reconstruction
X_pca_reconstructed = pca.inverse_transform(
    pca.transform(X_scaled)
)
# PPCA posterior mean reconstruction
# E[z|x] = (W.T W + sigma^2 I)^(-1) W.T (x - mu)
M = W.T @ W + sigma2 * np.eye(k)
Z_estimated = (X_scaled - mu) @ W @ np.linalg.pinv(M)
X_ppca_reconstructed = Z_estimated @ W.T + mu
pca_mse = np.mean((X_scaled - X_pca_reconstructed) ** 2)
ppca_mse = np.mean((X_scaled - X_ppca_reconstructed) ** 2)
print("\nRECONSTRUCTION ERROR")
print("Standard PCA MSE:", round(pca_mse, 4))
print("PPCA MSE:", round(ppca_mse, 4))
# 5. Compare reconstruction error for different components
components_list = range(1, X.shape[1])
pca_errors = []
ppca_errors = []
noise_estimates = []
for k in components_list:
```

```
    model = PCA(n_components=k)
    model.fit(X_scaled)
    # Standard PCA
    X_pca = model.inverse_transform(model.transform(X_scaled))
    pca_errors.append(np.mean((X_scaled - X_pca) ** 2))
    # PPCA model
    sigma2_k = max(model.noise_variance_, 1e-8)
    W_k = model.components_.T * np.sqrt(
        np.maximum(model.explained_variance_ - sigma2_k, 0)
    )
    M_k = W_k.T @ W_k + sigma2_k * np.eye(k)
    Z_k = (X_scaled - model.mean_) @ W_k @ np.linalg.pinv(M_k)
    X_ppca = Z_k @ W_k.T + model.mean_
    ppca_errors.append(np.mean((X_scaled - X_ppca) ** 2))
    noise_estimates.append(sigma2_k)
plt.figure(figsize=(8, 5))
plt.plot(list(components_list), pca_errors, marker="o",
         label="Standard PCA")
plt.plot(list(components_list), ppca_errors, marker="s",
         label="PPCA")
```

```
plt.xlabel("Number of Components")

plt.ylabel("Reconstruction MSE")

plt.title("PCA vs PPCA Reconstruction Error")

plt.grid(True)

plt.legend()

plt.show()



# 6. Plot estimated noise variance

plt.figure(figsize=(8, 5))

plt.plot(list(components_list), noise_estimates, marker="o")

plt.xlabel("Number of Components")

plt.ylabel("Estimated Noise Variance")

plt.title("PPCA Noise Variance Estimate")

plt.grid(True)

plt.show()

# 7. Generate new synthetic sensor samples from fitted PPCA

n_new = 10

# Draw latent variables z from a standard normal distribution

Z_new = np.random.randn(n_new, k)

# Generate Gaussian noise
```

```
epsilon = np.random.normal(
    0, np.sqrt(sigma2), size=(n_new, X.shape[1])
)

# Generate samples in standardized feature space
X_new_scaled = Z_new @ W.T + mu + epsilon
# Convert back to original sensor units
X_new = scaler.inverse_transform(X_new_scaled)
print("\n10 NEW SYNTHETIC SENSOR SAMPLES")
print(np.round(X_new, 2))
# 8. Visualize original and reconstructed sensor readings
sample_index = 0
plt.figure(figsize=(9, 5))
x_positions = np.arange(len(sensor_names))
width = 0.25
plt.bar(x_positions - width, X_scaled[sample_index],
        width, label="Original")
plt.bar(x_positions, X_pca_reconstructed[sample_index],
        width, label="Standard PCA")
plt.bar(x_positions + width, X_ppca_reconstructed[sample_index],
```

```
        width, label="PPCA")
plt.xticks(x_positions, sensor_names, rotation=20)
plt.ylabel("Standardized Sensor Value")
plt.title("Sensor Data Reconstruction Comparison")
plt.legend()
plt.tight_layout()
plt.show()
print("\nExperiment 20 completed successfully.")
```

OUTPUT:

```
Sensor data shape: (1000, 5)


NOISE VARIANCE ESTIMATION
----------------------------------------
Components: 1, Estimated noise variance: 0.3567
Components: 2, Estimated noise variance: 0.0946
Components: 3, Estimated noise variance: 0.0790
Components: 4, Estimated noise variance: 0.0745

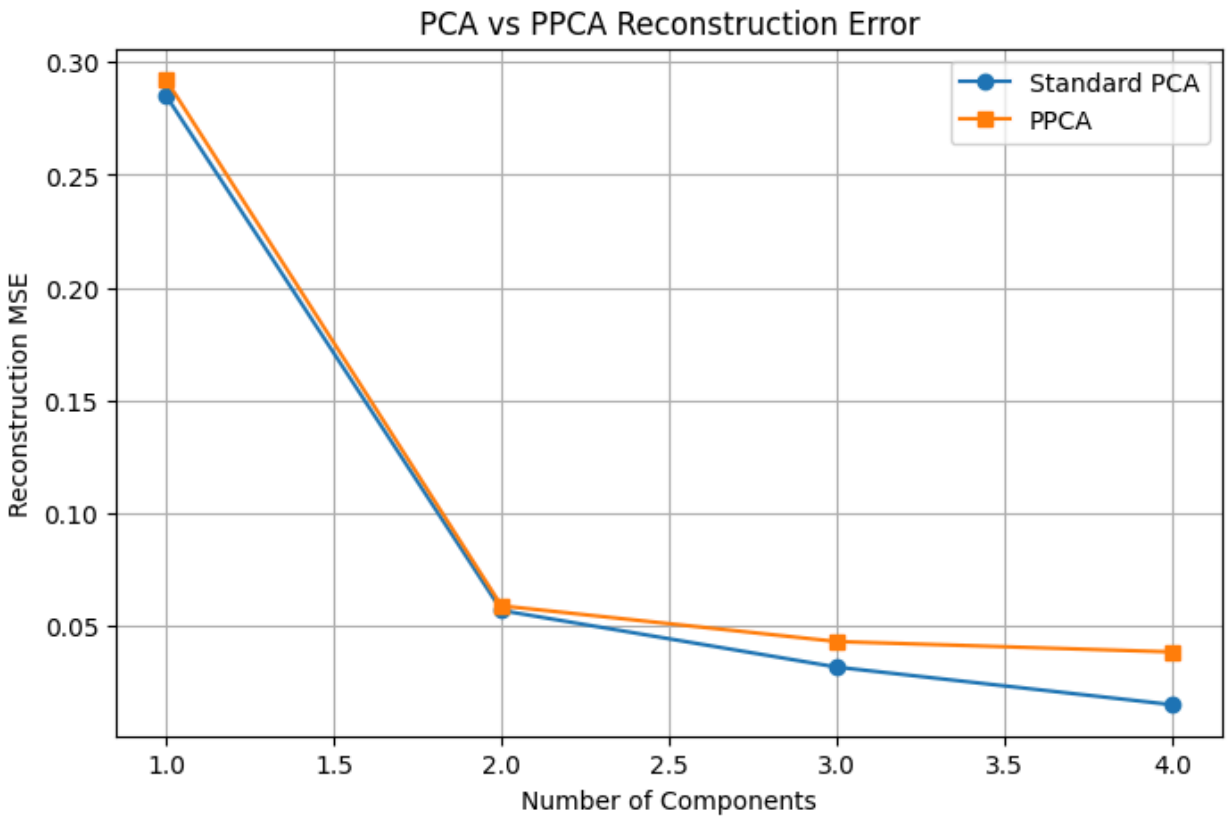
Selected components: 2
Estimated noise variance (sigma^2): 0.0946
Estimated noise standard deviation: 0.3075
```

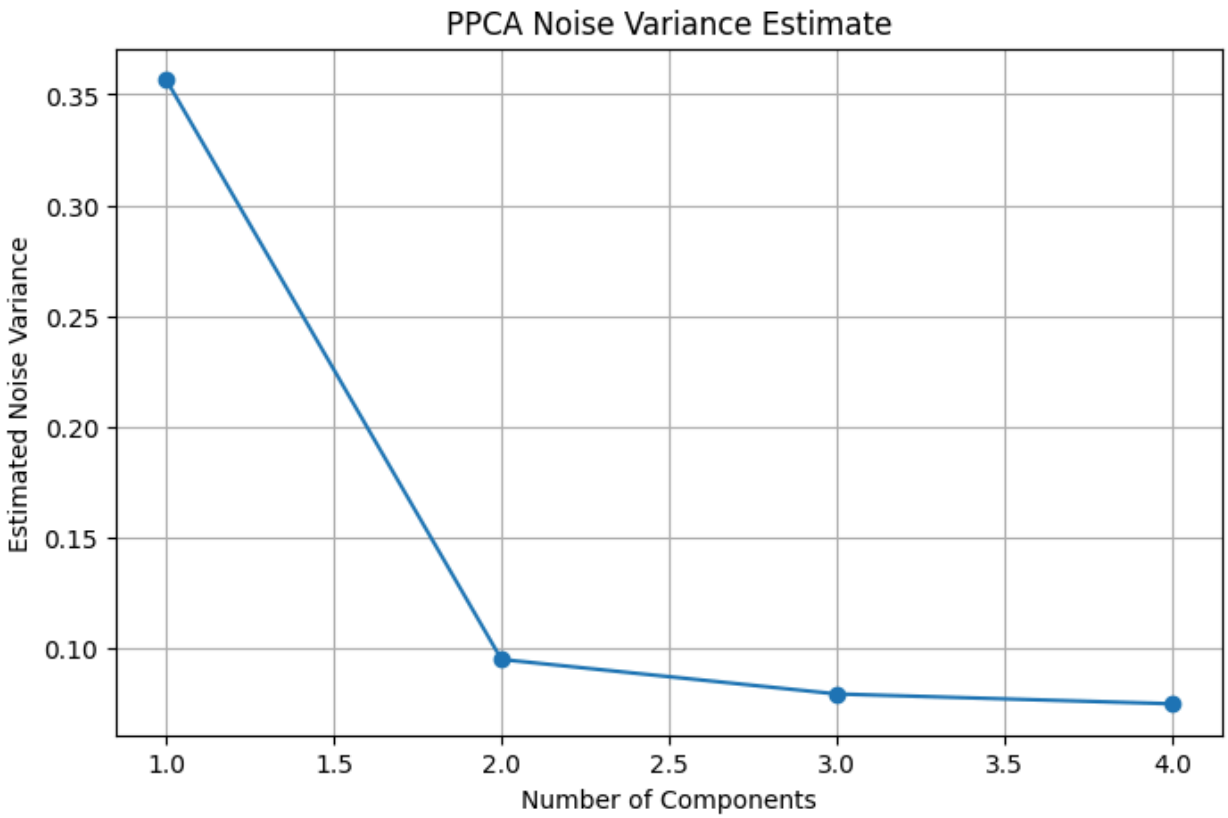
PPCA loading matrix shape: (5, 2)

RECONSTRUCTION ERROR

Standard PCA MSE: 0.0567

PPCA MSE: 0.0587





10 NEW SYNTHETIC SENSOR SAMPLES

[[29.38 6.64 98.66 9.79 61.66]

[26.79 6.31 100.17 10.29 61.11]

[24.73 5.11 102.51 11.16 61.54]

[27.35 5.33 103.72 12.2 61.63]

[26.54 5.48 104.33 12.3 61.95]

[25.22 5.41 100.53 9.85 59.95]

[26.52 5.81 103.06 12.03 61.15]

[23.11 4.22 100.14 10.97 59.92]

[26.79 5.96 98.49 9.42 60.19]

[26.66 6.49 102.02 12.74 60.82]]



.